electronics

MDPI

Article

RAP-RAG: A Retrieval-Augmented Generation Framework with Adaptive Retrieval Task Planning

Xu Ji *, Luo Xu, Landi Gu, Junjie Ma *, Zichao Zhang and Wei Jiang

Information Science Academy of China Electronics Technology Group Corporation, Beijing 100042, China
* Correspondence: 18840867169@163.com (X.J.); junjiema7237@gmail.com (J.M.)

Abstract

The Retrieval-Augmented Generation (RAG) framework shows great potential in terms of improving the reasoning and knowledge utilization capabilities of language models. However, most existing RAG systems heavily rely on large language models (LLMs) and suffer severe performance degradation when using small language models (SLMs), which limits their efficiency and deployment in resource-constrained environments. To address this challenge, we propose Retrieval-Adaptive-Planning RAG (RAP-RAG), a lightweight and high-efficiency RAG framework with adaptive retrieval task planning that is compatible with both SLMs and LLMs simultaneously. RAP-RAG is built on three key components: (1) a heterogeneous weighted graph index that integrates semantic similarity and structural connectivity; (2) a set of retrieval methods that balance efficiency and reasoning power; and (3) an adaptive planner that dynamically selects appropriate strategies based on query features. Experiments on the LiHua-World, MultiHop-RAG, and Hybrid-SQuAD datasets show that RAP-RAG consistently outperforms representative baseline models such as GraphRAG, LightRAG, and MiniRAG. Compared to lightweight baselines, RAP-RAG achieves 3–5% accuracy improvement while maintaining high efficiency and maintains comparable efficiency in both small and large model settings. In addition, our proposed framework reduces storage size by 15% compared to mainstream frameworks. Component analysis further confirms the necessity of weighted graphs and adaptive programming for robust retrieval under multi-hop reasoning and heterogeneous query conditions. These results demonstrate that RAP-RAG is a practical and efficient framework for retrieval-enhanced generation, suitable for large-scale and resource-constrained scenarios.

Keywords: retrieval-augmented generation; heterogeneous weighted graph; adaptive retrieval task planning; topology-enhanced retrieval

check for updates

Academic Editors: Phivos Mylonas, Katia Lida Kermanidis and Manolis Maragoudakis

Received: 25 September 2025
Revised: 27 October 2025
Accepted: 28 October 2025
Published: 30 October 2025

Citation: Ji, X.; Xu, L.; Gu, L.; Ma, J.; Zhang, Z.; Jiang, W. RAP-RAG: A Retrieval-Augmented Generation Framework with Adaptive Retrieval Task Planning. *Electronics* **2025**, *14*, 4269. https://doi.org/10.3390/electronics14214269

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

In recent years, with the development of Retrieval-Augmented Generation, the ability of large language models (LLMs) to retrieve and integrate external knowledge has been significantly improved [1–3]. Many valuable works have been completed from various research scenarios [4] such as knowledge quiz and machine translation. Current RAG frameworks demonstrate excellent performance thanks to their complex retrieval mechanisms and powerful language models. It is worth mentioning that, their heavy reliance on LLMs incurs significant computational overhead and resource consumption [5]. This issue is particularly acute in resource-constrained environments, significantly limiting their practical deployment capabilities [6]. Efficient resource deployment through SLMs is one of the current viable paths for AI application implementation. While these lightweight

models offer significant advantages in computational efficiency and deployment flexibility, they still face fundamental bottlenecks in the core operations of RAG particularly in key areas like semantic understanding and information retrieval. Currently mainstream RAG architectures [7,8] were initially designed to fully leverage the complex capabilities of LLMs. However, SLMs has inherent limitations in model capabilities. Replacing the LLMs in the existing RAG architecture with SLMs will result in severe performance degradation. The main reasons for this are as follows:

- **Query complexity**. For a query problem, a small model might not be able to handle very complex queries. Can a reasonable query method be constructed to match the performance of the small model? Furthermore, for problems of varying complexity, can the complexity be properly analyzed and the most appropriate method selected to search relevant databases, maximizing the performance of the small model?
- **Validation of the reasoning path**. For complex problems involving multi-step reasoning, the validity of the reasoning path must be ensured to retrieve the most relevant information for the query. For a small model, constructing a reasonable path using its own capabilities alone is difficult. Therefore, a method for validating the validity of the reasoning path is necessary to assist the small model in making this determination.

To address the above research issues, we propose RAP-RAG, a RAG framework for adaptive planning retrieval tasks. The construction of this framework is based on the following understanding of SLMs: (1) SLMs have limitations in understanding complex semantics, but can make a certain judgment on the complexity of a problem; (2) SLMs have limited ability to perform complex multi-step exploration, but if complex exploration is broken down into simple and well-defined steps, the system can maintain robustness under limited reasoning capabilities.

Based on the above understanding, our RAP-RAG framework proposes the following three key tasks:

1. **Heterogeneous Weighted Graph Index.** The construction of the knowledge base determines the retrieval quality and speed of the RAG system. In order to break through the limitations of SLMs in semantic capabilities, we constructed a heterogeneous weighted graph index. This design aims to provide a unified structure that supports semantic clustering and global relationship reasoning. Specifically, the index distinguishes and models two types of relationships: similarity and dissimilarity. Similarity relationships construct local communities based on score indicators to characterize the clustering characteristics of semantically related nodes; while dissimilar relationships describe the association strength between entities through weight indicators to maintain the overall topological structure and global connectivity. In this way, the graph index not only alleviates the problem of isolated semantic retrieval, but also enhances the model's ability to perform multi-hop reasoning and integrate global context, thereby providing comprehensive support for subsequent adaptive retrieval.
2. **Index-based Retrieval Method Set.** To enable reasonable retrieval of knowledge bases, we systematically constructed and modified a series of retrieval methods based on the heterogeneous weighted graph indexing mechanism, including vector retrieval, improved local retrieval, dual retrieval methods, and our proposed topology-enhanced retrieval method. These methods can fully utilize the semantic aggregation capabilities of local communities and combine graph structure information to perform complex relational reasoning, thereby achieving a better balance between different types of tasks.
3. **Adaptive Retrieval Task Planning.** For a query task, using a single retrieval method may not necessarily yield the best results. Instead, allowing the system to adaptively select the most appropriate retrieval method can be a solution. Therefore, we propose

a self-planning method for retrieval tasks. This method automatically selects or combines the most appropriate retrieval strategies based on the query requirements and question characteristics. For example, it prioritizes local retrieval in short-text question answering and topology-enhanced retrieval in multi-hop reasoning scenarios. This mechanism effectively improves the system's task generalization and usability in small-scale environments.

Through experiments on multiple task datasets, we verified the universality and efficiency of the RAP-RAG framework: Compared with existing RAG systems, RAP-RAG significantly improves retrieval efficiency and reasoning stability while maintaining high retrieval accuracy. Comparing the RAP-RAG framework's performance on LLMs and SLMs, RAP-RAG maintains robust performance, demonstrating superior lightweight features and resource adaptability in different application scenarios. This makes RAP-RAG not only suitable for general knowledge retrieval and reasoning tasks, but also meets the actual needs of local deployment, edge computing, and privacy-sensitive environments.

2. Previous Search

A growing body of research is exploring how to enhance the capabilities of language models through external knowledge retrieval, particularly in resource-constrained settings. This research encompasses three key areas: the rise of SLMs as effective alternatives to LLMs, the evolution of RAG frameworks, and the development of adaptive retrieval mechanisms that tailor search strategies to query characteristics. This section reviews representative progress in each area.

Small Language Models(SLMs). With the growing demand for lightweight and efficient AI systems, SLMs have become a research hotspot. Compared to large models, SLMs offer significant advantages in inference speed, resource consumption, and deployment flexibility, making them particularly suitable for edge devices and privacy-sensitive scenarios. In recent years, several open-source models have advanced this field, such as Llama-3.2-3B [9], Qwen2.5-1.5B [10], SmolLM-1.7B [11], Gemma-2-2B [12], and MobiLlama-1B [13]. These models demonstrate strong language understanding and reasoning capabilities while maintaining a very low parameter count and are easy to deploy in constrained computing environments. However, despite their excellent performance on general NLP tasks, effectively applying these models to complex tasks such as relation extraction remains an unexplored problem. This study attempts to propose new solutions in this area to address the shortcomings of current approaches.

Retrieval-Augmented Generation(RAG). Retrieval-augmented generation (RAG) systems enhance the capabilities of language models by retrieving relevant knowledge from external databases [5,7]. The core components of RAG are indexing, retrieval, and generation. The system first converts the original text set into a database and constructs an index for the database. It then retrieves relevant information based on the user query q. Finally, the retrieved information is fed into the language model as context to generate the answer. A common approach to database construction is to segment the text into searchable units [14,15] and store them in the database.

Furthermore, information is organized by constructing the text into a knowledge graph. Graph structures, as powerful tools for representing complex relationships, have been widely used in numerous fields. With the continued development of language models, researchers are increasingly interested in improving their ability to parse graph-structured data. GraphRAG [8] uses a language model combined with the Leiden algorithm to construct a graph-structured index, primarily for entity clustering. It also improves retrieval efficiency by generating community reports, integrating local and global information access, and relying on a unified retrieval mechanism. LightRAG [7] builds a two-tiered retrieval

architecture based on a knowledge graph. Its core innovation lies in decomposing queries into hierarchical components, from low-level details to high-level concepts. This enhances query comprehension capabilities, leading to more accurate document retrieval. MiniRAG [16] introduces heterogeneous graph indexing and lightweight retrieval mechanisms, demonstrating good performance on small models.

Adaptive Retrieval. To address differences in query complexity, the core approach is to dynamically decide whether to initiate a search based on the complexity. Mallen et al. [17] determined complexity based on entity frequency, enabling search only for low-frequency queries. However, this approach only makes a binary decision of "retrieve or not," making it difficult to handle complex queries with multi-step reasoning. Qi et al. [18] based on the BERT model, repeated the fixed "retrieve-read-rerank" process until an answer was found. However, this approach failed to account for differences in query complexity and required additional model training, resulting in low adaptability. Self-RAG [19] proposed training a complex model to achieve an integrated "retrieval-analysis-generation" process. However, a single model struggles to adapt to diverse queries—it's either too simple to handle complex queries or too complex, wasting resources. Adaptive-RAG uses a classifier to distinguish query difficulty: simple questions (the answer is often in a single document) perform a single query, retrieve the document, and integrate the information to generate the answer [20,21]. Complex questions (multi-step reasoning) require multiple calls to the model and retriever [22,23]. However, this method's retrieval strategy is relatively simple and still has room for optimization.

3. Materials and Methods

In this section, we introduce the detailed architecture of our proposed RAP-RAG framework. As shown in Figure 1, RAP-RAG consists of three key components: (1) Heterogeneous Weighted Graph Index (Section 3.1), which simultaneously models semantic similarity and structural integrity to achieve unified knowledge representation; (2) Index-based Retrieval Method Set (Section 3.2), which integrates multiple retrieval strategies to balance efficiency and accuracy; and (3) Adaptive Retrieval Task Planning (Section 3.3), which adaptively selects or combines retrieval methods based on query features.

3.1. Heterogeneous Weighted Graph Index

During graph construction, entities can be prone to duplication. Entities with high semantic similarity may be scattered across multiple nodes, leading to noise accumulation, edge redundancy, and weakened community semantics. This problem is particularly acute in small models, whose semantic reasoning capabilities are inherently limited compared to larger models. Therefore, we propose an entity aggregation mechanism and, within this mechanism, a method for constructing heterogeneous weighted graphs. The construction process is illustrated in Figure 2. During the basic graph construction process, the weights of relationships between entities are stored as "Weight". During the subsequent entity aggregation process, entities with similarity are connected using "Similar" relationships, with the weights stored as "Score."

Existing graph construction methods still face challenges such as handling complex knowledge graph attributes and requiring high semantic capabilities [7,8,16]. In contrast, our method employs a multistage aggregation pipeline that integrates embedding-level candidate generation, structural entity aggregation via weakly connected components, and a hybrid textual–attribute similarity filter. This design not only mitigates the risks of over-merging and under-merging but also enables small models to construct heterogeneous, weighted graphs with high fidelity. In essence, aggregation is elevated from a peripheral operation to a core enabler of precise graph topology, ensuring that knowledge integra-

tion, reasoning, and retrieval remain both robust and scalable—even under constrained model capacities.

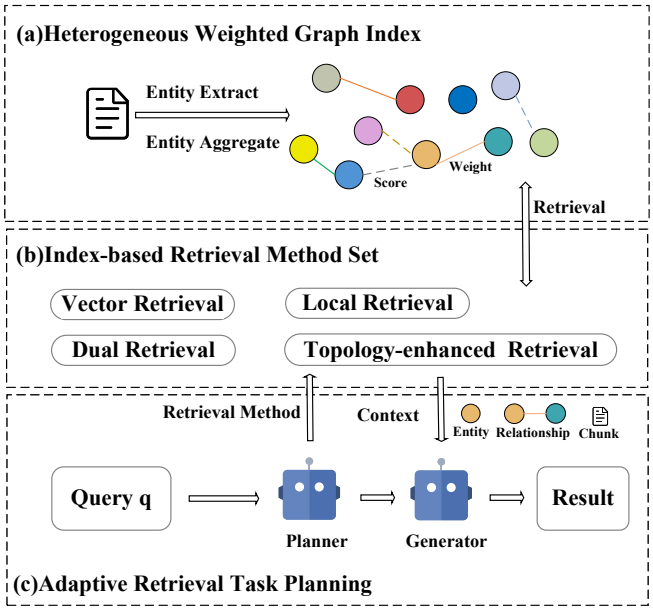


Figure 1. RAP-RAG. The framework consists of three main components: (**a**) Heterogeneous Weighted Graph Index, which constructs entities through entity extraction and aggregation, and organizes them according to similarity scores and relationship weights to support subsequent retrieval; (**b**) Index-based Retrieval Method Set, which includes vector retrieval, local retrieval, dual retrieval, and topology-enhanced retrieval, allowing for flexible invocation of different retrieval strategies; and (**c**) Adaptive Retrieval Task Planning, in which a planner selects the appropriate retrieval method based on the query task and drives the generator to produce results based on contextual information, achieving integrated retrieval and generation processing.

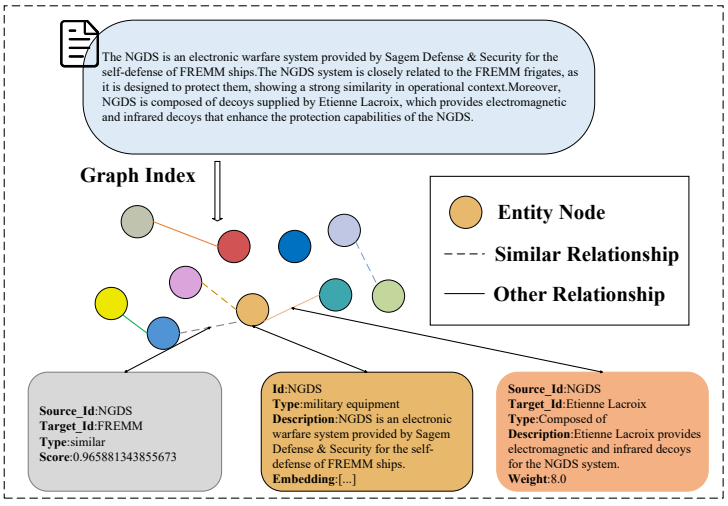


Figure 2. Heterogeneous Weighted Graph Index. Text information is extracted through entity extraction to obtain entity nodes, which are then connected through different types of relationships. Dotted lines represent similar relationships, reflecting the similarity of entities in semantics or operational scenarios; solid lines represent other semantic relationships.

To ensure robust and accurate entity aggregation, we propose a multi-stage filtering mechanism that combines embedding similarity, structural aggregation, and hybrid similarity measures. This approach mitigates the limitations of single-threshold methods by

progressively refining candidate entities through four steps: candidate generate, entity aggregate, textual and attribute-level filter, and final merge.

(1) Candidate Generate

Given an entity e_i with embedding vector v_i, its candidate set $N(e_i)$ is constructed using cosine similarity with a threshold τ:

$$N(e_i) = \{e_j \mid \cos(v_i, v_j) \geq \tau,\ 0 \leq \tau \leq 1\}. \tag{1}$$

where e_j and v_j represent existing entities in the knowledge base and their corresponding embedding vectors. Usually the value of τ is greater than 0.7. This step ensures that only semantically related entities are considered for further evaluation.

(2) Entity Aggregate

To capture structural consistency among entities, we apply weakly connected components (WCC) clustering over the candidate graph $G = (V, E)$, where edges are defined by embedding-based similarity:

$$\mathcal{P} = \{P_1, P_2, \ldots, P_k, \ldots, P_N\}, \quad P_k \subseteq V, \tag{2}$$

where N is the number of components detected by WCC, $\mathcal{P}$ is the set of all connected components, and each P_k represents a group of entities that may refer to the same object. This step groups entities into coarse communities for subsequent fine-grained analysis.

(3) Textual And Attribute Filter

For each pair of entities $e_i, e_j \in P_m$, we compute a hybrid similarity score that integrates the textual edit distance and the attribute level similarity:

$$S(e_i, e_j) = \lambda \cdot \text{EditSim}(l_i, l_j) + (1 - \lambda) \cdot \text{AttrSim}(a_i, a_j), \quad \lambda \in [0, 1], \tag{3}$$

where l_i, l_j denote entity labels, a_i, a_j denote attribute sets, and

$$\text{EditSim}(l_i, l_j) = 1 - \frac{\text{EditDist}(l_i, l_j)}{\max(|l_i|, |l_j|)}. \tag{4}$$

Here, EditDist is the Levenshtein distance, and AttrSim can be instantiated with Jaccard or cosine similarity computed via APOC operators (Neo4j Graph Data Science Library, Neo4j Inc., San Mateo, CA, USA). This hybrid formulation balances surface-form similarity with structural attribute consistency.

(4) Final Merge

Entities e_i and e_j are merged if their hybrid similarity score exceeds a predefined threshold δ:

$$R_{sim} = \{(e_i, e_j) \mid S(e_i, e_j) \geq \delta,\ 0 \leq \delta \leq 1\} \tag{5}$$

This guarantees that only entities with strong textual and semantic agreement are unified, thereby minimizing both over-merging and under-merging.

The multistage entity aggregation process described above is the cornerstone of our heterogeneous weighted graph index. By gradually refining the candidate set through embedding similarity, structural aggregation, and mixed text attribute filtering, this mechanism ensures that graph nodes are semantically coherent and structurally consistent. This enables higher embedding robustness to noise and improved scalability within limited

model capacity. This lays a solid foundation for subsequent tasks such as knowledge integration, reasoning, and retrieval.

3.2. Index-Based Retrieval Method Set

To achieve adaptive retrieval task planning, we built a set of index-based retrieval methods M based on heterogeneous weighted index graphs.

$$M = \{\text{VR}, \text{LR}, \text{DR}, \text{TR}\} \tag{6}$$

This set of methods includes four retrieval paradigms: Vector Retrieval, Local Retrieval, Dual Retrieval, and Topology-enhanced Retrieval. The following is a detailed introduction to these four search methods.

(1) Vector Retrieval

In our work, vector retrieval operates directly using text blocks as the basic retrieval unit. Specifically, the original corpus is first divided into several text blocks based on semantics and discourse structure. These blocks are then converted into high-dimensional vector representations using a pre-trained embedding model and stored in the index. When a user initiates a query, the system performs the same vectorization processing on the query content, embedding it in the same semantic space. The system then calculates the similarity between the query vector and all text block vectors and ranks the candidate results based on the score, prioritizing the text blocks that are semantically closest to the query as retrieval results.

$$C_{VR} = V(\mathcal{E}(q)) \tag{7}$$

In vector retrieval, q represents the user's natural language query, and $\mathcal{E}(q)$ represents its embedding vector representation. The function $V(\cdot)$ performs a similarity search on the embedding vector within the vector database, resulting in a set of candidate chunks C_{VR}. This formula describes the basic process of vector retrieval: first, the query is converted into a dense semantic embedding, then a nearest neighbor search is performed in the index space to obtain a set of candidate entities that are semantically relevant to the query.

(2) Local Retrieval

Local retrieval uses a vector database to search for relevant entities in the database and uses them as search terms. It then extracts the relationships and textual information related to the entities and maps them to the corresponding entities, extracting all the content as contextual information and inputting it into the SLMs.The local retrieval process is shown in Figure 3.

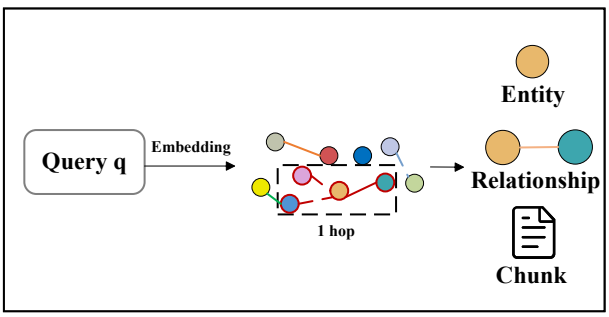


Figure 3. Local Retrieval. The natural language query q is first transformed into its embedding representation, which is used to search the vector index. The retrieved entities are expanded via one-hop traversal in the knowledge graph, enabling the extraction of relevant entities, relationships, and associated text chunks.

First, the user's natural language query q is transformed into a dense semantic embedding $\mathcal{E}(q)$, and a similarity search operation $V(\cdot)$ is applied over the vector database to retrieve the candidate entity set:

$$E_{LR} = V(\mathcal{E}(q)) \tag{8}$$

This step performs semantic-based local entity retrieval. Subsequently, in order to enrich the contextual information, the retrieved entity set E_{LR} is expanded within the knowledge graph G by performing a one-hop neighborhood search. This process yields the associated relation set R_{LR}:

$$R_{LR} = G_{1\text{-}hop}(E_{LR}) \tag{9}$$

After retrieving the one-hop neighborhood, we obtain the relation set $R_{LR} = R_{LR}^{sim} \cup R_{LR}^{str}$, where R_{sim} denotes semantic similarity relations (e.g., entity alignment or embedding-based proximity), and R_{str} denotes structural relations (e.g., functional predicates from the knowledge graph). Both connect to the retrieved entity set E_{LR}.

To balance semantic relevance and structural diversity, we select relations proportionally from each subset. Let α be the desired proportion of semantic relations, and $1 - \alpha$ the proportion of structural relations. We select the top $\lfloor \alpha k \rfloor$ relations from R_{LR}^{sim} and the top $\lceil (1-\alpha)k \rceil$ relations from R_{LR}^{str}, then combine them into the final candidate subset:

$$R_{LR}^{*} = \text{Top-}\lfloor \alpha k \rfloor \left(R_{LR}^{sim} \right) \cup \text{Top-}\lceil (1-\alpha)k \rceil \left(R_{LR}^{str} \right). \tag{10}$$

Here, α can be set empirically (e.g., 0.7) or adapted based on dataset characteristics. This ensures both semantic and structural signals are adequately represented in the context, preventing dominance by any single relation type. The selected relation set R_{LR}^{*}, along with corresponding entities and textual snippets, forms the enhanced context input to the SLMs.

The final context block C_{LR} is constructed by retrieving all textual descriptions related to entities in E_{LR} and relations in R_{LR}^{*} from the knowledge graph G using the text index function $G_{index}(\cdot)$:

$$C_{LR} = G_{index}(E_{LR}, R_{LR}^{*}) \tag{11}$$

Finally, E_{LR}, R_{LR}^{*}, and C_{LR} are obtained as context inputs to the SLMs.

(3) Dual Retrieval

Dual Retrieval further bases the search process on the explicit entities and relationships extracted from the query. It first extracts low-level and high-level keywords, then returns the corresponding entities, relationships, and chunks based on a heterogeneous weighted graph according to predefined hyperparameters. The dual retrieval mechanism is illustrated in Figure 4.

Here, $\mathcal{K}(\cdot)$ is a keyword extraction function, K_{low} represents fine-grained keywords emphasizing specific entities and detailed concepts, whereas K_{high} represents abstract keywords capturing the main topic and relational context.

$$K_{low}, K_{high} = \mathcal{K}(q) \tag{12}$$

Next, the low-level and high-level keywords are converted into embeddings and searched within the graph G to obtain the candidate entity set E_{DR} and the candidate relation set R_{DR}.

$$E_{DR} = G(\mathcal{E}(K_{low})) \tag{13}$$

$$R_{DR} = G(\mathcal{E}(K_{high})) \tag{14}$$

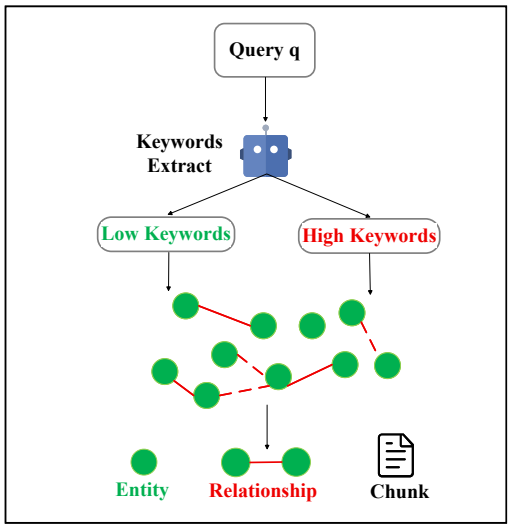


Figure 4. Dual Retrieval. Low-level and high-level keywords are extracted from the query q. Low-level keywords focus on specific entities and details, while high-level keywords focus on the main idea and subject. When executing the query, high-level queries search for the most relevant edges based on relationships, while low-level queries search for the most relevant nodes based on entities. Ultimately, the entities, relationships, and related text blocks are returned.

At this stage, the heterogeneous weighted graph we constructed shows that the relations corresponding to R_{DR} are structural relations and do not include similar relations. Therefore, we denote this relation set as R_{DR}^{str}:

$$R_{DR}^{str} = R_{DR} \tag{15}$$

Since relying solely on structural relations may overlook semantically relevant connections, we use the top-ranked subset of similar relations R_{DR}^{sim} to expand the relation set. Specifically, we select $k - n$ relations from R_{LR}^{sim}, where n is the number of R_{DR}^{str} and k is the total number of relations to be selected, and merge them with R_{DR}^{str} to form a mixed relation set R_{DR}^{*}:

$$R_{DR}^{*} = R_{DR}^{str} \cup \text{Top-}\lfloor k - n \rfloor \left(R_{LR}^{sim} \right) \tag{16}$$

The last step is the same as local retrieval, retrieving all chunks related to the entities in E_{DR} and the relations in R_{DR}^{*} from the knowledge graph G through $G_{index}(\cdot)$:

$$C_{DR} = G_{index}(E_{DR}, R_{DR}^{*}) \tag{17}$$

(4) Topology-enhanced Retrieval

For complex problems, SLMs often introduce a significant amount of noise during the retrieval process due to their inherently limited capabilities, preventing them from performing deep reasoning based on their own capabilities. Therefore, we propose a topology-enhanced retrieval method that effectively combines semantic and structural information from heterogeneous weighted graphs. An overview of the topology-enhanced retrieval process is given in Figure 5. This method first uses vector search to filter initial entities and answer entity type predictions. This then proceeds to the topology enhancement phase, where the most important reasoning paths are identified by calculating entity relevance scores and type prediction scores within the heterogeneous weighted graph. Entity, relationship, and chunk information are then derived from these reasoning paths. Our method achieves high-precision knowledge retrieval while maintaining computa-

tional efficiency, ultimately providing more accurate and interpretable reasoning paths for augmented generation tasks.

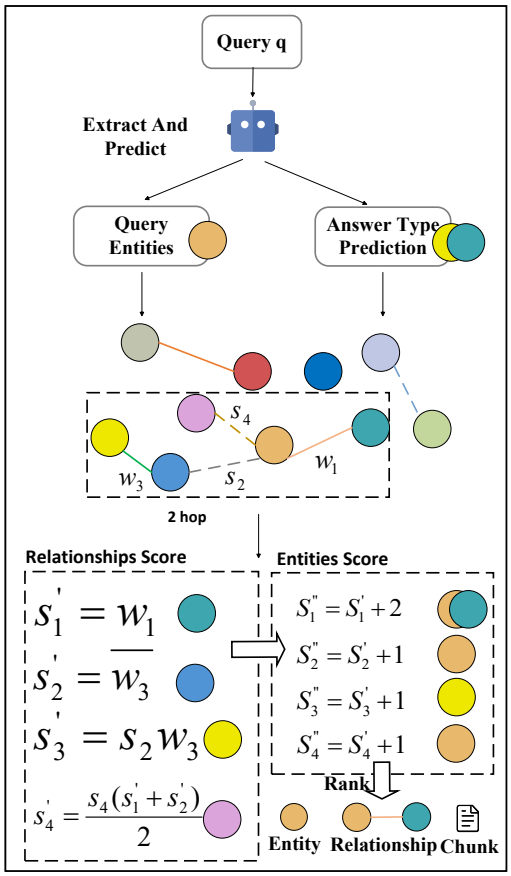


Figure 5. Topology-enhanced Retrieval.

For a query q, the entity recognition module first extracts explicitly appearing objects, generating an input entity set K_e, which characterizes the entity nodes directly involved in the query. Simultaneously, the system performs type prediction on the query intent, generating a type prediction set K_a, whose elements correspond to potential answer categories or attribute constraints. The combination of these two provides structured input conditions for subsequent retrieval and filtering:

$$K_e, K_a = \mathcal{K}(q) \tag{18}$$

Based on the extracted entity set K_e and the type prediction set K_a, we further project them into a heterogeneous weighted graph G for entity retrieval. Specifically, the entities explicitly appearing in the query are mapped to E^e_{TR}, while the predicted answer type node is E^a_{TR}:

$$E^e_{TR} = G(\mathcal{E}(K_e)) \tag{19}$$

$$E^a_{TR} = G(\mathcal{E}(K_a)) \tag{20}$$

Before constructing the candidate relationship set, the retrieved entity sets E^e_{TR} and E^a_{TR} are used as anchor points for a k-hop traversal over the knowledge graph G. This process expands their neighborhood to include semantically relevant relationships, yielding the candidate relationship set R_{TR}:

$$R_{TR} = G_{k\text{-}hop}(E^e_{TR} \cup E^a_{TR}) \tag{21}$$

After the candidate relationship set R_{TR} is constructed, the system further evaluates and filters the entities and relationships it contains. Specifically, each candidate relationship relies not only on the edge weight information S_R within the knowledge graph, but also incorporates the query semantics and the predicted answer types to compute the entity score S_E.

Relationships Score S_R: In the process of calculating the relationship score S_E, since there are two weights, Score and Weight, in the heterogeneous weighted graph, the process of calculating the relationship score needs to be divided into three cases for discussion.

(1) Directly connected to the query entity through other relationships. For the case where other relations are directly connected to the query entity, the weight parameter is directly used as the relation score:

$$S_R(r) = \omega_r \tag{22}$$

where ω_r is the weight corresponding to the relationship $r \in R_{TR}$. For directly connected entities, the original weight is used directly instead without further calculation.

(2) Directly connected to other entities through similar relationships. Directly connected to other entities through similar relationships. In the case of direct connection to other entities through similar relationships, if the tail entity has other relationship edges, the weight average of all relationship edges is directly calculated and multiplied by the score of the similar relationship edges. If the tail entity has no other relationship edges, the mean of the head entity relationship scores is calculated and multiplied by the score of the similar relationship edges. The specific formula is as follows:

$$S_R(r) = \begin{cases} s_r \cdot \dfrac{1}{|G_r(e_t)|} \displaystyle\sum_{r' \in G_r(e_t)} \omega_{r'} & \text{if } N_r(e_t) \neq \emptyset, \\ s_r \cdot \dfrac{1}{|G_r(e_h)|} \displaystyle\sum_{r' \in G_r(e_h)} s_{r'} & \text{otherwise,} \end{cases} \tag{23}$$

where $G_r(e)$ is the set of other relations excluding the relations connected to entity e, e_h is the head entity, e_t is the tail entity, and s_r is the score of similar relations.Note that the head entity here refers to the starting entity of the query, and the tail entity refers to the ending entity of the query, which is different from the head and tail entities with directions in the usual sense.

(3) Indirectly connected.

For a 2-hop indirect connection, the score of the indirect connection can be obtained by multiplying the comprehensive weights on the 2-hop path:

$$S_R(r) = w(r_1) \cdot w(r_2) \tag{24}$$

where $w(\cdot)$ denotes the unified weight of relation, defined as

$$w(r) = \begin{cases} \omega_r & \text{if } r \in R_{str}, \\ s_r & \text{if } r \in R_{sim}. \end{cases} \tag{25}$$

This multiplicative formulation captures the compositional relevance of multi-hop paths: a path is only as strong as its weakest link. By using the unified weight function $w(r)$, we seamlessly integrate structural (R_{str}) and semantic (R_{sim}) relations within heterogeneous reasoning paths. Note that for longer paths (>2 hops), the score can be extended recursively

as the product of all edge weights along the path. In practice, we restrict to 2-hop paths to balance expressiveness and computational efficiency.

Entities Score S_E:

During the computation of the entity score S_E, each edge in R_{TR} is traversed to determine whether the entities it connects are the starting entity or the predicted answer entity. In theory, an edge that directly connects the starting entity and the answer entity along the shortest path is highly relevant to the query q.S_E is calculated as follows:

$$S_E(r) = \sum_{e \in E^e_{TR}} \text{count}(e, r) + \sum_{e \in E^a_{TR}} \text{count}(e, r) \tag{26}$$

where $r \in R_{TR}$ is an edge in the relation set, and $\text{count}(e, r)$ is a counting function that counts the number of entities that satisfy the requirement. This formula assigns higher scores to relations that connect query anchor entities or answer entities, thereby promoting edges that bridge the semantic gap between questions and their potential answers.

Finally, the two scores are added together to form the final score, and the most relevant path is selected after filtering according to the score, and all entities, relationships and chunks along the path are extracted.

$$S(r) = S_r(r) + S_e(r) \tag{27}$$

We then select the top k relations with the highest scores and get the entities connected to them:

$$E'_{TR}, R'_{TR} = \text{Top-}k(E_{TR}, R_{TR}\,;\, S) \tag{28}$$

Finally, the relevant chunks are as follows:

$$C_{TR} = G_{index}(E'_{TR}, R'_{TR}) \tag{29}$$

3.3. Adaptive Retrieval Task Planning

To maximize the capabilities of SLMs, we developed an adaptive retrieval task planning method based on a heterogeneous weighted graph and a set of retrieval methods. The core concept of this method is to use a task-driven approach to enable the model to autonomously select the most appropriate retrieval method based on the semantic features and structural requirements of the query, thereby improving the overall effectiveness of retrieval enhancement generation.An example of this adaptive planning process is shown in Figure 6.

Specifically, we utilize a prompt word design and a task planning mechanism to guide the model to dynamically invoke the optimal retrieval method in different scenarios. The prompt word not only explicitly describes the task type but also provides a detailed description of the characteristics of different retrieval methods based on the characteristics of the retrieval method set. This allows the model to simultaneously consider semantic similarity and knowledge structure characteristics, achieving accurate discrimination and method matching for complex problems.

Based on this, we designed a fine-tuning dataset construction method. By assigning hierarchical labels to datasets in different retrieval modes and task scenarios, and gradually increasing the retrieval complexity by judging whether the generated results meet the requirements at each level, the SLMs exhibits strong task adaptability and generalization capabilities even with a limited parameter size.

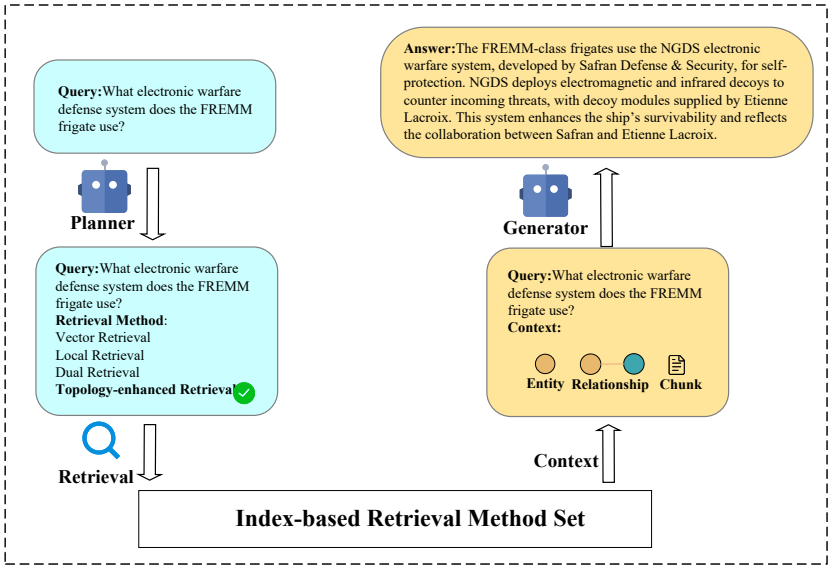


Figure 6. Adaptive Retrieval Task Planning.The user submits a query. The planner selects the optimal method from retrieval method set based on the task characteristics and constructs context by combining entities, relations, and semantic fragments. The generator then uses the retrieved context to generate an answer, producing high-quality, task-oriented results.

(1) Prompt Construction

In our research, the core goal of hints is to help SLMs automatically select the optimal retrieval method based on query characteristics. We designed the planner's hints from the following perspectives.For specific examples of prompt words, please see Appendix A.

Number and complexity of entities. When a query contains only a single entity and is relatively focused, answers can often be generated directly without external search, or matching can be achieved solely based on semantic similarity. Therefore, direct generation or vector search is more suitable. When a query involves multiple entities and requires knowledge integration within a specific scope, local search is preferred to ensure targeted results.

Question scope. For closed questions, the answer often exists in a single document fragment or training corpus. Such questions are suitable for direct generation or vector search. However, when the question explicitly targets a database, a specific context, or a local knowledge subset, hints classify it as a local search. If the question spans multiple hierarchies and involves multiple levels of reasoning, dual search is needed to obtain information hierarchically. For complex questions that require inferring answers from global relationships, topology-enhanced search should be enabled.

Depth of reasoning. If a query requires only a shallow answer and does not involve a reasoning chain, it can be solved directly through generation or single-step vector matching. When a query requires multi-hop reasoning, if the reasoning chain is limited to a specific subgraph or database, the prompt word guides the model to choose local search. If the reasoning spans different levels or information sources, dual search is more suitable. For complex reasoning tasks that require multiple hops and involve global connections in the graph structure, the prompt word explicitly recommends topology-enhanced search to fully utilize the structural information of the graph.

Relational complexity. When the knowledge connection between the query and the answer is weak or relies only on single-point information, direct generation or vector search can meet the needs. When appropriate combination of local knowledge points is required, local search is selected. When the query involves hierarchical knowledge connections, such as inferring global conclusions from local facts, the prompt word recommends dual search.

In cases where complex relationships across the entire topological network need to be retrieved and a global reasoning chain needs to be constructed, the prompt word explicitly points to topology-enhanced search. This hierarchical design allows for more refined query classification and ensures the rationality and robustness of the method selection.

(2) Planner Fine-tuning Strategy

In adaptive retrieval task planning, effectively training the planner to accurately select the optimal retrieval method is key to improving system performance. However, existing datasets lack explicit annotations tailored to the requirements of different retrieval methods. Therefore, we propose an automated dataset construction and fine-tuning strategy. For details on the fine-tuning dataset examples, see Appendix B.

Specifically, we first design a label generation mechanism based on a multi-retrieval method set. For the same query q, we employ four methods for inference and generation: vector retrieval, local retrieval, two-level retrieval, and topology-enhanced retrieval. Labels are determined based on the quality of the results. If the query can be directly generated without retrieval, it is assigned label "A"; if the simplest vector retrieval yields the correct answer, the query is assigned label "B"; if a local retrieval is required, it is assigned label "C"; if a more complex two-level retrieval is required to obtain the correct answer, it is assigned label "D"; and if the correct answer requires topology-enhanced retrieval, it is assigned label "E". This approach ensures that the labels reflect the complexity requirements of the query under different methods.

However, this strategy also has limitations: some queries fail to generate the correct answer under all retrieval methods and therefore receive no labels. To address this issue, we further incorporate the inherent structural bias of the dataset for supplementary annotation. For example, queries from single-hop question-answering datasets that are not labeled in the first phase are assigned a default label of "C," whereas queries from multi-hop reasoning datasets are assigned an "E." This supplementary mechanism significantly improves data annotation coverage while maintaining rationality.

Based on the automatically generated query-method label pairs, we construct a training set for fine-tuning the planner. Subsequently, we fine-tune the SLMs using the cross-entropy loss function to learn the correspondence between different queries and optimal retrieval strategies. During the inference phase, given a new query q, the planner predicts its optimal retrieval strategy, enabling adaptive selection of retrieval paths.

This combination of automated annotation and fine-tuning not only effectively learns the applicable conditions for different retrieval methods but also significantly improves the adaptability and generalization of the SLMs in complex task scenarios.

4. Experiment

Based on the RAP-RAG framework described above, our experiments mainly consist of the following two parts: (1) Performance comparison. How does RAP-RAG compare with existing state-of-the-art solutions in terms of retrieval accuracy and efficiency? (2) Component analysis. How do heterogeneous weighted graphs and various retrieval methods contribute to the overall performance of the RAP-RAG framework? To what extent does adaptive retrieval task planning improve performance?

4.1. Datasets

To the effectiveness of the proposed RAP-RAG framework, we select three datasets with varying levels of query complexity and retrieval difficulty: LiHua-World [16], MultiHop-RAG [24], and Hybrid-SQuAD [25]. The reason for choosing these datasets lies in their complementarity: LiHua-World provides realistic conversational queries with fragmented and evolving contexts, MultiHop-RAG evaluates complex multi-hop reasoning.

across documents, and Hybrid-SQuAD integrates both simple and complex queries into a unified benchmark. Together, they enable comprehensive validation of our framework across diverse retrieval scenarios.

LiHua-World. LiHua-World is a benchmark constructed from one year of simulated conversational records of a virtual user. It spans diverse aspects of daily life such as social interactions, sports, entertainment, and personal events. The dataset includes single-hop, multi-hop, and summary-style queries, each annotated with gold answers and supporting evidence. Its conversational and temporal characteristics make it suitable for evaluating retrieval planning in fragmented and evolving information environments.

MultiHop-RAG. MultiHop-RAG is designed to assess complex reasoning that requires gathering evidence across multiple documents. Each query is paired with gold answers and supporting passages, where the relevant information is often distributed and demands sequential reasoning to connect. This dataset highlights the necessity of dual-stage retrieval and topology-enhanced retrieval, and serves as a benchmark for testing robustness in multi-hop reasoning tasks.

Hybrid-SQuAD. Hybrid-SQuAD extends the classical SQuAD v1.1 dataset by integrating both single-hop and multi-hop question-answer pairs. It deliberately combines simple factoid-style questions with more challenging compositional queries. This hybrid structure allows systematic evaluation of RAP-RAG across varying query complexities and demonstrates the benefits of adaptive retrieval task planning compared to fixed retrieval strategies.

4.2. Baselines

We compare RAP-RAG against several representative retrieval-augmented generation baselines:

NaiveRAG [26]. NaiveRAG adopts a straightforward retrieval-augmented generation paradigm. It directly encodes queries and documents into the same embedding space, and retrieves candidate passages using similarity-based nearest neighbor search. This approach provides a strong lower-bound baseline for evaluating more advanced retrieval strategies.

GraphRAG [8]. GraphRAG incorporates structured knowledge into the retrieval process by constructing graphs from unstructured text. Nodes represent entities or semantic units, while edges capture their relations. Retrieval is guided not only by semantic similarity but also by graph connectivity, enabling reasoning over contextual relationships beyond surface-level embeddings.

LightRAG [7]. LightRAG introduces an efficient layered retrieval design. It first identifies coarse-grained semantic regions within the knowledge space, and then refines retrieval at a fine-grained level. This hierarchical retrieval reduces unnecessary search space and improves efficiency while maintaining high accuracy on complex queries.

MiniRAG [16]. MiniRAG is a lightweight framework designed for constrained environments. It focuses on modularity and efficiency by supporting plug-and-play language models and embedding functions. Through compact indexing and streamlined retrieval pipelines, MiniRAG offers competitive performance with significantly lower resource consumption, making it suitable for scenarios with limited computational budgets.

4.3. Models

In our research, the planner and generator are supported by the same SLMs. For the model selection, we employ different configurations for LLMs and SLMs. In the advanced LLMs setting, we use gpt-4o-mini as the language model and text-embedding-3-small as the specialized embedding model. For the lightweight SLMs setting, we utilize the optimized

text embedding-3-small as embedding model, paired with selected SLMs including Llama-3.2-3B [9], Qwen2.5-1.5B [10], and MobiLlama-1B [13].

4.4. Performance Comparison

We first conducted performance analysis experiments on three datasets. The specific experimental results are shown in Table 1. The results clearly demonstrate that RAP-RAG outperforms baseline methods in accuracy on the LiHuaWorld, MultiHop-RAG, and Hybrid-SQuAD benchmarks. Compared to MiniRAG, RAP-RAG consistently achieves improvements of approximately 3–5 percentage points across all test settings, demonstrating that the proposed adaptive retrieval planner and heterogeneous graph index improve retrieval quality without significantly increasing computational overhead.

Table 1. Performance evaluation using accuracy (%). Higher values indicate better RAG performance. Results compare baseline methods against RAP-RAG across multiple datasets. Bold values indicate best performance, while "/" denotes cases where methods failed to generate valid responses.

Dataset	Model	Accuracy (%)				
		NaiveRAG	GraphRAG	LightRAG	MiniRAG	RAP-RAG
LiHuaWorld	Llama-3.2-3B	41.37	/	39.86	53.41	**58.29**
	Qwen2.5-1.5B	42.83	/	35.79	52.63	**57.52**
	MobiLlama-1B	43.68	/	39.25	48.83	**53.71**
	gpt-4o-mini	46.62	35.41	56.97	54.23	**61.18**
MultiHop-RAG	Llama-3.2-3B	42.65	/	27.19	49.87	**54.71**
	Qwen2.5-1.5B	44.37	/	/	51.36	**56.14**
	MobiLlama-1B	39.52	/	21.95	48.63	**53.41**
	gpt-4o-mini	53.74	60.88	64.95	68.37	**71.29**
Hybrid-SQuAD	Llama-3.2-3B	35.19	/	30.27	42.73	**46.61**
	Qwen2.5-1.5B	34.46	/	29.57	41.44	**47.38**
	MobiLlama-1B	33.25	/	28.16	39.75	**44.63**
	gpt-4o-mini	45.63	48.24	51.39	54.18	**59.13**

Accuracy values in %. Bold indicates best performance per row.

Existing state-of-the-art RAG systems, such as LightRAG and GraphRAG, experience severe performance degradation or even failure when using SLMs, while RAP-RAG maintains reliable accuracy. On challenging datasets such as Hybrid-SQuAD, RAP-RAG equipped with an SLM consistently outperforms MiniRAG, demonstrating its robustness in resource-constrained settings where advanced LLMs are unavailable.

This improvement in accuracy is attributed to RAP-RAG's unique adaptive retrieval mechanism. RAP-RAG effectively balances semantic and structural information by dynamically choosing between vector-based, topology-enhanced, and local retrieval strategies, achieving robust performance even in multi-hop reasoning tasks like MultiHop-RAG. This design reduces over-reliance on pure language generation capabilities and improves the reliability of the retrieval enhancement process.

The Figure 7 illustrates the trade-off between accuracy and storage requirements across different RAG systems. As shown, RAP-RAG achieves the highest accuracy of 57.52% with a storage size of only 16 MB, representing merely a 1 MB increase compared to MiniRAG (52.63%). This demonstrates that the proposed adaptive retrieval and heterogeneous graph indexing design effectively enhance accuracy without introducing significant storage overhead. By contrast, LightRAG consumes a much larger storage footprint of 19 MB yet suffers from severe accuracy degradation (35.79%), while GraphRAG, even with the support of gpt-4o-mini, delivers similarly poor accuracy (35.41%) under higher storage costs. These results highlight a critical advantage of RAP-RAG: it maintains efficiency

comparable to lightweight methods like MiniRAG while consistently outperforming them in accuracy, and simultaneously avoids the inefficiency and instability issues observed in existing advanced systems. Overall, RAP-RAG achieves a superior balance of accuracy and efficiency, making it particularly suitable for deployment in resource-constrained environments where both performance and storage matter.

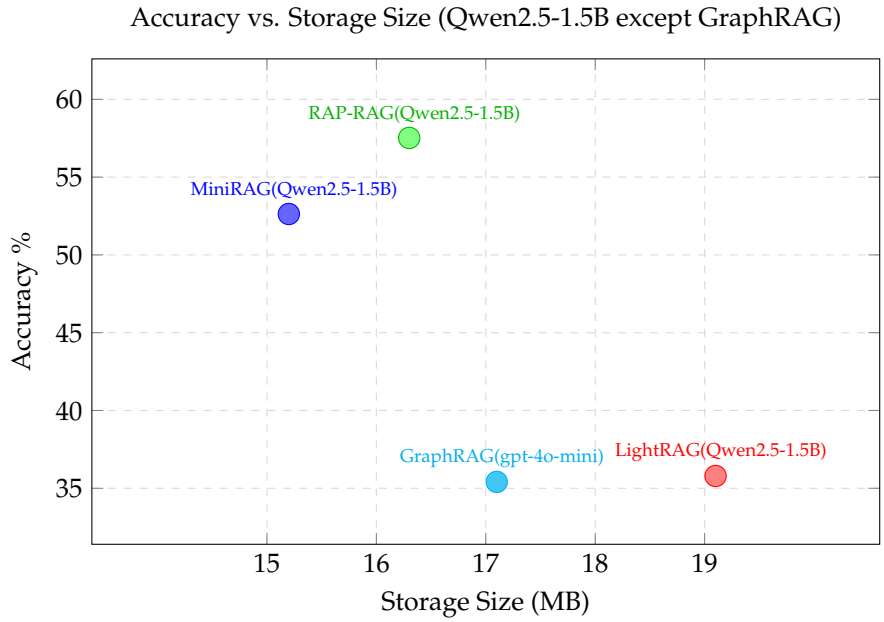


Figure 7. Accuracy-storage trade-off for different RAG methods. Except for GraphRAG (using gpt-4o-mini), all methods use Qwen2.5-1.5B. RAP-RAG achieves the highest accuracy with only 1 MB storage overhead over MiniRAG.

4.5. Component Analysis

To evaluate the contributions of different components of the RAP-RAG framework to overall performance, we conducted three ablation experiments on the Hybrid-SQuAD dataset. Accuracy and query efficiency were used as the two main metrics. All experiments used the full RAP-RAG framework as a control.

In Table 2, we removed graph weights for comparison. This removal degenerates the retrieval method, eliminating the need to distinguish between similar relations and other relations. This simplifies the weight calculation to a cumulative count of key edges and nodes in the path. The Ablation experiment 1 results show that removing graph weights leads to a decrease in accuracy across all datasets and model configurations. For example, on the MultiHop-RAG dataset, Llama-3.2-3B experienced a 5.7% drop in accuracy, while Qwen2.5-1.5B experienced a 6.1% drop in accuracy, demonstrating that weighted edges are crucial for complex multi-hop reasoning tasks. A similar trend was observed on the LiHuaWorld dataset, where the Base configuration outperformed the -Weights variant by over 5% across all SLMs. On the Hybrid-SQuAD dataset, despite lower overall accuracy due to the mix of simple and complex queries, the Base system still demonstrated a significant advantage, improving by approximately 4% to 5% over the -Weights variant.

These findings demonstrate that incorporating weighted and heterogeneous relations into graph indexes is crucial for guiding effective retrieval. By leveraging similarity scores and edge weights, RAP-RAG prioritizes more relevant entities and contextual links, thereby improving accuracy without sacrificing efficiency. In particular, the improvements achieved with MultiHop-RAG confirm that weighted graph structures are particularly beneficial for multi-step reasoning scenarios.

Table 2. Ablation experiment 1: Accuracy comparison between full RAP-RAG (Base) and variant without graph weights (-Weights). Bold indicates best performance per row.

Dataset	Model	Accuracy (%)	
		Base	-Weights
LiHuaWorld	Llama-3.2-3B	**58.29**	52.11
	Qwen2.5-1.5B	**57.52**	51.82
	MobiLlama-1B	**53.71**	48.96
	gpt-4o-mini	**61.18**	58.33
MultiHop-RAG	Llama-3.2-3B	**54.71**	49.01
	Qwen2.5-1.5B	**56.14**	50.31
	MobiLlama-1B	**53.41**	47.69
	gpt-4o-mini	**71.29**	65.17
Hybrid-SQuAD	Llama-3.2-3B	**46.61**	41.22
	Qwen2.5-1.5B	**47.38**	42.17
	MobiLlama-1B	**44.63**	39.79
	gpt-4o-mini	**59.13**	54.03

Accuracy values in %. "Base" = full RAP-RAG (copied from Table 1); "-Weights" = remove edge weights in heterogeneous graph.

Table 3 results confirm three key observations. First, the adaptive planner consistently improves accuracy across most datasets and model settings, with performance gains ranging from 2–4% compared to single-method baselines, at the cost of slightly higher latency due to the additional planning overhead. Second, fixed strategies such as Vector or Local achieve lower latency but suffer from reduced accuracy, indicating that relying on a single retrieval mechanism limits robustness. Third, the Topology-based strategy performs competitively and even surpasses the Base configuration in the MultiHop-RAG setting, suggesting that structural information is especially important for multi-hop reasoning tasks. Overall, these results demonstrate that the adaptive planning mechanism in RAP-RAG consistently improves accuracy across diverse settings. This improvement comes with a modest latency increase of typically two to four tenths of a second. The trade-off is justified in accuracy-sensitive applications, where even small gains of two to four percentage points can significantly impact user experience. For latency-critical scenarios, fixed strategies such as Local offer viable alternatives, albeit at the cost of reduced robustness.

Table 3. Ablation experiment 2: Comparison between full RAP-RAG (Base) with adaptive planner and fixed-strategy variants.

Dataset	Model	Accuracy%/Latency (s)				
		Base	Vector	Local	Dual	Topology
LiHuaWorld	Llama-3.2-3B	**58.3**/1.42	55.1/1.08	52.4/0.95	53.8/1.21	57.8/1.15
	Qwen2.5-1.5B	**57.5**/1.45	54.2/1.05	51.8/0.93	52.7/1.18	56.9/1.12
	MobiLlama-1B	**53.7**/1.31	50.6/0.97	48.3/0.89	49.5/1.09	52.8/1.03
	gpt-4o-mini	**61.2**/1.56	58.1/1.21	55.6/1.05	56.8/1.34	60.7/1.27
MultiHop-RAG	Llama-3.2-3B	54.7/1.65	51.3/1.19	48.6/1.03	49.8/1.42	**56.1**/1.36
	Qwen2.5-1.5B	56.1/1.69	52.6/1.16	49.7/1.01	51.0/1.39	**59.3**/1.32
	MobiLlama-1B	53.4/1.58	49.8/1.09	47.1/0.98	48.3/1.29	**56.2**/1.24
	gpt-4o-mini	71.3/1.84	67.9/1.34	64.5/1.17	65.8/1.58	**72.1**/1.49
Hybrid-SQuAD	Llama-3.2-3B	**46.6**/1.22	43.5/0.92	40.8/0.81	41.9/1.05	46.0/0.99
	Qwen2.5-1.5B	**47.4**/1.24	44.1/0.90	41.3/0.79	42.5/1.03	46.8/0.97
	MobiLlama-1B	**44.6**/1.15	41.0/0.85	38.2/0.76	39.3/0.98	43.7/0.92
	gpt-4o-mini	**59.1**/1.36	55.2/1.01	52.3/0.89	53.6/1.17	58.5/1.12

Values are Accuracy%/Average Query Latency (s). "Base" = full RAP-RAG (with adaptive planner). Fixed variants disable planner and always use a single retrieval method. Bold indicates best performance per row.

5. Discussion

Our RAP-RAG framework outperforms existing retrieval-augmented generation (RAG) baselines on all three datasets, demonstrating the advantages of combining heterogeneous weighted graph indexing with adaptive retrieval task planning. Compared to MiniRAG, RAP-RAG achieves approximately 3–5 percentage points in accuracy improvement, while incurring only a slight increase in computational overhead. This demonstrates that the design of the weighted graph structure and adaptive retrieval strategy are crucial for enabling SLMs to handle complex reasoning tasks without sacrificing efficiency.

While a 15% storage reduction may seem insignificant, it can be significant in large-scale or resource-constrained deployments. On mobile or IoT devices with limited storage, reducing index size by 15% can help improve battery life and enable the simultaneous deployment of multiple models. In a cloud environment serving millions of users, saving 4MB per instance can add up to terabytes of storage savings across the system. For private, on-premises RAG systems, smaller indexes can improve load times and reduce hardware requirements.

Mainstream frameworks suffer from instability or inefficiency when deployed with SLMs, RAP-RAG maintains robust performance across both large and small models. This robustness demonstrates that adaptive retrieval task planning is a key step in bridging the gap between resource-intensive LLMs systems and lightweight, deployable solutions suitable for environments with limited computational resources. Despite these encouraging results, several limitations remain. First, while the adaptive planner improves retrieval robustness, it introduces additional latency, which can hinder real-time applications. Secondly, the retrieval method set we constructed includes four retrieval methods. However, their design primarily relies on summarizing existing methods and proactively designing the planner's retrieval strategy. However, existing methods may not cover all cases, and the retrieval method set leaves room for further improvement.

Furthermore, although our evaluation covers three representative datasets, they are all English datasets due to quality issues, and their applicability on multilingual datasets requires further verification. Furthermore, more extensive testing in professional fields such as biomedicine and law is needed to verify its universality. When constructing fine-tuning datasets, existing label annotation methods are typically designed according to a hierarchical principle. However, in practical applications, the methods used do not progress from simple to complex, but rather there may be multiple retrieval methods of similar complexity, or even a mixture of multiple retrieval methods. Therefore, the label annotation method needs to be optimized together with the set of retrieval methods.

In summary, the discussion emphasizes that RAP-RAG successfully advances the RAG framework by balancing accuracy, efficiency, and adaptability. Future research should continue to refine adaptive planning and expand the retrieval method set to fully unleash the potential of lightweight retrieval-augmented generation systems.

6. Conclusions

In this paper, we propose RAP-RAG, a novel retrieval-augmented generation framework that combines weighted heterogeneous graph indexing with adaptive retrieval task planning. This framework aims to improve the robustness and efficiency of inference, particularly for deployments with SLMs in resource-constrained environments. We validate the effectiveness of RAP-RAG through systematic experiments on three benchmark datasets: LiHuaWorld, MultiHop-RAG, and Hybrid-SQuAD. Results show that our approach consistently outperforms existing mainstream frameworks (including NaiveRAG, GraphRAG, LightRAG, and MiniRAG) in terms of accuracy, while maintaining good efficiency and scalability in terms of latency and storage overhead.

Further component analysis and ablation experiments demonstrate that the key components of RAP-RAG contribute significantly to the performance improvements. The weighted heterogeneous graph index significantly enhances accuracy in multi-hop reasoning scenarios, while the adaptive retrieval task planning effectively addresses the limitations of single retrieval strategies, which can be unstable in complex problems. In contrast, while some fixed strategies have a slight advantage in latency, they lag significantly behind in accuracy and generalization, demonstrating the necessity and effectiveness of adaptive planning mechanisms. We believe that RAP-RAG provides a solid foundation for efficient and scalable retrieval-enhanced generation research and will play an important role in promoting the coordinated development of large and small models.

Author Contributions: Conceptualization, X.J.; Methodology, X.J.; Software, X.J.; Validation, X.J.; Investigation, L.X.; Data curation, L.X., L.G. and J.M.; Writing—original draft, X.J.; Writing—review and editing, L.G. and J.M.; Visualization, L.X.; Supervision, Z.Z. and W.J.; Project administration, L.G. and J.M.; Funding acquisition, Z.Z. and W.J. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the National Natural Science Foundation of China (U23B200380, U23B200539).

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: Authors Xu Ji, Luo Xu, Landi Gu, Zichao Zhang, Junjie Ma, Wei Jiang were employed by the company Information Science Academy of China Electronics Technology Group Corporation.The authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

Appendix A

To enable SLMs to automatically select the most suitable retrieval strategy, we design structured prompts that explicitly describe the available methods and the corresponding decision rules. These prompts guide the planner to classify queries into one of five methods based on entity complexity, reasoning depth, and relational scope. The example below illustrates the standardized prompt template used in our experiments.

Table A1. Prompts for adaptive retrieval task planning.

```
You are a search strategy planner.
Given a user query, your task is to determine which search method is most appropriate.
Available methods include:
1. Direct Generation (A): Obtain the answer directly from the model without searching.
2. Vector Retrieval (B): Retrieve semantically similar text blocks using embedding
    similarity.
3. Local Retrieval (C): Retrieve from a specific database, domain, or local knowledge
    subset.
4. Dual Retrieval (D): Perform hierarchical search first searching for high-level
    context at a high level, then searching for details at a high level.
5. Topology-Enhanced Retrieval (E): Uses graph-based reasoning and multi-hop
    traversal across global relationships.
Decision Rule:
- If the query involves **a single entity** or is **simple and fact-based**, choose
    Direct Generation (A) or Vector Search (B), based on the difficulty of the
    question and the complexity of the entity.
- If the query involves **multiple entities in the same domain or database**, choose
    Local Search (C).
- If the query spans **multi-layer reasoning** or **hierarchical knowledge**, select
    Dual Search (D).
- If the query requires **complex multi-hop reasoning** or **global connections**,
    select Topology Enhanced Search (E).
Output Format:
The answer contains only one method option: A, B, C, D, or E.
The user's query is as follows:
{query}
```

Appendix B

These samples provides a representative sample of the fine-tuning dataset designed for the adaptive retrieval task planner in the RAP-RAG framework, aiming to supplement the dataset construction details not fully elaborated in the main text. The sample follows the "query-direct generation attempt-multi-method retrieval verification-final judgment" workflow, which is consistent with the automated dataset labeling mechanism of RAP-RAG—i.e., assigning retrieval method labels (A/B/C/D/E) by comparing the validity of results from different retrieval strategies.

Table A2. Fine-Tuning Dataset Samples.

Question: Who was the child of César Gaytan, the Italian navigator who explored the continent's eastern coast? Was he born to an English parent?
Direct Generation: Cesare Gaitán was born on the North American continent. The Italian navigator who explored the east coast of North America for the British was Giovanni Caboto, also known as John Cabot. So the answer is: Giovanni Caboto/John Cabot. **Answer**: Wrong
.....
Topoloy-enhanced Search:César Gaitán was born in Guadalajara, Jalisco, Mexico. The Italian navigator who explored the east coast of the continent for the British was John Cabot. John Cabot's son was Sebastian Cabot. So the answer is: Sebastian Cabot. **Answer**: Right **Label**: E

References

1. Salemi, A.; Zamani, H. Evaluating Retrieval Quality in Retrieval-Augmented Generation. In Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, Washington, DC, USA, 14–18 July 2024; Association for Computing Machinery: New York, NY, USA, 2024; pp. 2395–2400. [CrossRef]
2. Es, S.; James, J.; Espinosa Anke, L.; Schockaert, S. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, St. Julian's, Malta, 17–22 March 2024; Aletras, N., De Clercq, O., Eds.; Association for Computational Linguistics: St. Julians, Malta, 2024; pp. 150–158. [CrossRef]
3. Sudhi, V.; Bhat, S.R.; Rudat, M.; Teucher, R. RAG-Ex: A Generic Framework for Explaining Retrieval Augmented Generation. In Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, Washington, DC, USA, 15–18 June 2024; Association for Computing Machinery: New York, NY, USA, 2024; pp. 2776–2780. [CrossRef]
4. Fan, W.; Ding, Y.; Ning, L.-B.; Wang, S.; Li, H.; Yin, D.; Chua, T.-S.; Li, Q. A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models. In Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Barcelona, Spain, 25–29 August 2024; pp. 1234–1245. [CrossRef]
5. Gao, Y.; Xiong, Y.; Gao, X.; Jia, K.; Pan, J.; Bi, Y.; Dai, Y.; Sun, J.; Wang, H. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv* **2023**, arXiv:2312.10997.
6. Liu, Z.; Zhao, C.; Iandola, F.; Lai, C.; Tian, Y.; Fedorov, I.; Xiong, Y.; Chang, E.; Shi, Y.; Krishnamoorthi, R.; et al. MobileLLM: Optimizing Sub-Billion Parameter Language Models for On-Device Use Cases. In Proceedings of the Forty-First International Conference on Machine Learning, Vienna, Austria, 21–27 July 2024.
7. Guo, Z.; Xia, L.; Yu, Y.; Ao, T.; Huang, C. LightRAG: Simple and Fast Retrieval-Augmented Generation. *arXiv* **2025**, arXiv:2410.05779.
8. Edge, D.; Trinh, H.; Cheng, N.; Bradley, J.; Chao, A.; Mody, A.; Truitt, S.; Larson, J. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv* **2024**, arXiv:2404.16130.
9. Grattafiori, A.; Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Vaughan, A., et al. The Llama 3 Herd of Models. *arXiv* **2024**, arXiv:2407.21783.
10. Qwen, A.Y.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Li, C.; Liu, D.; Huang, F., et al. Qwen2.5 Technical Report. *arXiv* **2025**, arXiv:2412.15115.
11. Allal, L.B.; Lozhkov, A.; Bakouch, E.; Blázquez, G.M.; Penedo, G.; Tunstall, L.; Marafioti, A.; Kydlíček, H.; Lajarín, A.P.; Srivastav, V., et al. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model. *arXiv* **2025**, arXiv:2502.02737.
12. Gemma Team, M.R.; Pathak, S.; Sessa, P.G.; Hardin, C.; Bhupatiraju, S.; Hussenot, L.; Mesnard, T.; Shahriari, B.; Ramé, A.; Ferret, J., et al. Gemma 2: Improving Open Language Models at a Practical Size. *arXiv* **2024**, arXiv:2408.00118.

13. Thawakar, O.; Vayani, A.; Khan, S.; Cholakal, H.; Anwer, R.M.; Felsberg, M.; Baldwin, T.; Xing, E.P.; Khan, F.S. MobiLlama: Towards Accurate and Lightweight Fully Transparent GPT. *arXiv* **2024**, arXiv:2402.16840.
14. Mao, Y.; He, P.; Liu, X.; Shen, Y.; Gao, J.; Han, J.; Chen, W. Generation-Augmented Retrieval for Open-Domain Question Answering. *arXiv* **2020**, arXiv:2009.08553.
15. Qian, H.; Zhang, P.; Liu, Z.; Mao, K.; Dou, Z. MemoRAG: Moving Towards Next-Gen RAG via Memory-Inspired Knowledge Discovery. *arXiv* **2024**, arXiv:2409.05591.
16. Fan, T.; Wang, J.; Ren, X.; Huang, C. MiniRAG: Towards Extremely Simple Retrieval-Augmented Generation. *arXiv* **2025**, arXiv:2501.06713.
17. Mallen, A.; Asai, A.; Zhong, V.; Das, R.; Khashabi, D.; Hajishirzi, H. When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Toronto, ON, Canada, 9–14 July 2023; Association for Computational Linguistics: Toronto, ON , Canada, 2023; pp. 9802–9822. Available online: https://aclanthology.org/2023.acl-long.556 (accessed on 27 October 2025).
18. Qi, P.; Lee, H.; Sido, T.; Manning, C.D. Answering Open-Domain Questions of Varying Reasoning Steps from Text. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP 2021), Virtual Event, 7–11 November 2021; Association for Computational Linguistics: Punta Cana, Dominican Republic, 2021; pp. 3599–3614. Available online: https://aclanthology.org/2021.emnlp-main.285 (accessed on 27 October 2025).
19. Asai, A.; Wu, Z.; Wang, Y.; Sil, A.; Hajishirzi, H. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024), Vienna, Austria, 7–11 May 2024. Available online: https://openreview.net/forum?id=hSyW5go0v8 (accessed on 27 October 2025).
20. Lazaridou, A.; Gribovskaya, E.; Stokowiec, W.; Grigorev, N. Internet-Augmented Language Models through Few-Shot Prompting for Open-Domain Question Answering. *arXiv* **2022**, arXiv:2203.05115.
21. Ram, O.; Levine, Y.; Dalmedigos, I.; Muhlgay, D.; Shashua, A.; Leyton-Brown, K.; Shoham, Y. In-Context Retrieval-Augmented Language Models. *Trans. Assoc. Comput. Linguist.* **2023**, *11*, 1316–1331. [CrossRef]
22. Press, O.; Zhang, M.; Min, S.; Schmidt, L.; Smith, N.A.; Lewis, M. Measuring and Narrowing the Compositionality Gap in Language Models. In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, 6–10 December 2023. Available online: https://aclanthology.org/2023.findings-emnlp.745 (accessed on 27 October 2025).
23. Trivedi, H.; Balasubramanian, N.; Khot, T.; Sabharwal, A. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Toronto, ON, Canada, 9–14 July 2023; Association for Computational Linguistics: Toronto, ON, Canada, 2023; pp. 10014–10037. Available online: https://aclanthology.org/2023.acl-long.571 (accessed on 27 October 2025).
24. Tang, Y.; Yang, Y. MultiHop-RAG: Benchmarking Retrieval-Augmented Generation for Multi-Hop Queries. *arXiv* **2024**, arXiv:2401.15391.
25. Taffa, T.A.; Banerjee, D.; Assabie, Y.; Usbeck, R. Hybrid-SQuAD: Hybrid Scholarly Question Answering Dataset. *arXiv* **2024**, arXiv:2412.02788.
26. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-T.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 9459–9474. Available online: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf (accessed on 27 October 2025).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.